Playwright Lab Quick Start Manual

1. Purpose

This manual provides a quick orientation to the Playwright lab environment. It explains where the lab applications are located, how to open the applications under test, and how learners can install and set up their own Playwright project.

This guide is intended for lab access and basic Playwright setup. Assignment execution, detailed test design, defect analysis, and submission preparation will be covered separately.

---

2. Accessing the Lab

1. Log in to TalentFarm.
2. Open the main menu from the top-right corner.
3. Select **Labs**.
4. Start your assigned lab VM.
5. Connect to the VM using RDP.

The Playwright lab runs on a Windows Server VM. After the VM is started or restarted, RDP may take a few minutes to become available.

---

3. Opening PowerShell

Once connected to the VM:

1. Open **PowerShell**.
2. Use PowerShell to navigate folders, inspect the application, and install Playwright.

---

4. Main Lab Folder

The lab files are located under:

```
C:\qa-lab
```

In PowerShell, run:

```
cd C:
dir
```

You should see a folder named:

```
qa-lab
```

Open the lab folder:

```
cd C:\qa-lab
dir
```

You should see folders similar to:

```
apps
db-dumps
logs
playwright
tools
```

---

5. Important Lab Folders

Folder	Purpose
`C:\qa-lab\apps`	Contains the applications used for testing
`C:\qa-lab\logs`	Contains lab or application logs
`C:\qa-lab\db-dumps`	Contains database dump or reset-related files
`C:\qa-lab\tools`	Contains supporting lab tools
`C:\qa-lab\playwright`	May contain reference or lab-support Playwright files, if provided

Learners should not assume that Playwright is already fully installed or ready to use. Learners are expected to create and use their own Playwright project.

---

6. Application Folders

The applications under test are located in:

```
C:\qa-lab\apps
```

Run:

```
cd C:\qa-lab\apps
dir
```

You should see folders similar to:

```
finserve-retail-demo
insurance-order-entry-demo
lab-landing
```

Folder	Purpose
`finserve-retail-demo`	FinServe demo application
`insurance-order-entry-demo`	Insurance order entry demo application
`lab-landing`	Lab landing or navigation application

These are the web applications learners will inspect and automate using Playwright.

---

7. Opening the Applications in Browser

The lab applications are served through the Windows Server VM.

Common local URLs inside the VM are:

```
http://127.0.0.1:8081
http://127.0.0.1:8082
```

Common login URLs may be:

```
http://127.0.0.1:8081/login
http://127.0.0.1:8082/login
```

Open these URLs in a browser inside the VM and identify which application is running on each port.

If accessing from outside the VM, use the VM public IP:

```
http://<vm-public-ip>:8081/login
http://<vm-public-ip>:8082/login
```

Replace `<vm-public-ip>` with the public IP address of your assigned VM.

---

8. Confirm the Applications Are Responding

You can verify the application pages from PowerShell:

```
curl.exe -I http://127.0.0.1:8081/login
curl.exe -I http://127.0.0.1:8082/login
```

Expected result:

```
HTTP/1.1 200 OK
```

A `200 OK` response confirms that the application page is reachable.

If the browser does not load but the local curl command works, the issue may be related to browser, firewall, public access, or network configuration.

---

9. Create Your Own Playwright Project

Learners should create their own Playwright project under their user folder.

Go to the learner folder:

```
cd C:\Users\talentfarm_learner
```

Create a new folder for Playwright work:

```
mkdir playwright-assignment
cd playwright-assignment
```

This folder will be used for your Playwright installation and test files.

---

10. Confirm Node.js and npm

Playwright requires Node.js and npm.

Run:

```
node -v
npm -v
```

Expected result:

```
Node.js version is displayed
npm version is displayed
```

If both commands return version numbers, Node.js and npm are available.

If these commands are not recognized, raise it as a lab setup issue before continuing.

---

11. Install Playwright

From your project folder:

```
C:\Users\talentfarm_learner\playwright-assignment
```

run:

```
npm init playwright@latest
```

When prompted, use the assignment or mentor instructions. If no specific instruction is given, a typical setup is:

```
Language: JavaScript
Test folder: tests
GitHub Actions workflow: No
Install Playwright browsers: Yes
```

After installation is complete, confirm Playwright:

```
npx playwright --version
```

Expected result:

```
Version <number>
```

---

12. Review the Playwright Project Files

After installation, run:

```
dir
```

You should see files and folders similar to:

```
tests
test-results
playwright-report
node_modules
package.json
package-lock.json
playwright.config.js
```

Folder / File	Purpose
`tests`	Playwright test files
`test-results`	Screenshots, videos, traces, and failure artifacts
`playwright-report`	HTML test report generated after test execution
`node_modules`	Installed Node.js dependencies
`package.json`	Project dependencies and scripts
`package-lock.json`	Locked dependency versions
`playwright.config.js`	Playwright configuration file

Run Playwright commands from your own project folder:

```
C:\Users\talentfarm_learner\playwright-assignment
```

---

13. Check Available Tests

To list the tests available in your project, run:

```
npx playwright test --list
```

This command displays the test files and test cases Playwright can detect.

If no tests are found, check that:

```
You are inside your Playwright project folder
The test files are inside the tests folder
The file names follow Playwright naming conventions
The Playwright configuration file is present
```

Common valid test file names include:

```
example.spec.js
login.spec.js
insurance-login.spec.js
finserve-login.spec.js
```

---

14. Run a Basic Playwright Test

To run all tests in the project:

```
npx playwright test
```

To run tests only on Chromium:

```
npx playwright test --project=chromium
```

To run a specific test file:

```
npx playwright test tests\<test-file-name>.spec.js --project=chromium
```

Replace `<test-file-name>` with the actual test file name.

Example:

```
npx playwright test tests\example.spec.js --project=chromium
```

---

15. Run a Test in Headed Mode

Headed mode opens the browser while the test is running.

Use this when you want to visually observe what Playwright is doing:

```
npx playwright test --project=chromium --headed
```

To run one test file in headed mode:

```
npx playwright test tests\<test-file-name>.spec.js --project=chromium --
headed
```

Example:

```
npx playwright test tests\example.spec.js --project=chromium --headed
```

---

16. Create a Simple Application Smoke Test

After confirming the application opens in the browser, learners can create a simple smoke test.

Create a new file:

```
notepad .\tests\app-smoke.spec.js
```

Paste the following example:

```
const { test, expect } = require('@playwright/test');

test('Application login page loads', async ({ page }) => {
  await page.goto('http://127.0.0.1:8081/login');

  await expect(page).toHaveURL(/login/);
  await expect(page.locator('body')).toBeVisible();
});
```

Save the file.

Run the test:

```
npx playwright test tests\app-smoke.spec.js --project=chromium --headed
```

If your assigned application is on port `8082`, update the URL:

```
await page.goto('http://127.0.0.1:8082/login');
```

This smoke test confirms that Playwright can open the application page.

---

17. Generate the HTML Report

To generate an HTML report, run:

```
npx playwright test --project=chromium --reporter=html
```

Then open the report:

```
npx playwright show-report
```

Important note:

If you run tests using the list reporter:

```
npx playwright test --reporter=list
```

Playwright shows results in the terminal, but it may not generate an HTML report. Use `--reporter=html` when you want to view the report in the browser.

---

18. Viewing Test Results

After a test run, check the following folders:

```
dir .\test-results
dir .\playwright-report
```

The `test-results` folder may contain screenshots, videos, traces, and failure artifacts.

The `playwright-report` folder contains the HTML test report.

These files are useful for understanding test failures and collecting evidence.

---

19. Application Logs

Application logs are available under:

```
C:\qa-lab\logs
```

To view the logs folder:

```
cd C:\qa-lab\logs
dir
```

Logs may help when troubleshooting application issues or collecting evidence for defects.

---

20. Quick Playwright Setup Checklist

Use this checklist when starting the Playwright lab:

```
1. Open PowerShell.
2. Review application folders:
   cd C:\qa-lab\apps
   dir

3. Open the applications in the browser:
   http://127.0.0.1:8081/login
   http://127.0.0.1:8082/login

4. Go to learner folder:
   cd C:\Users\talentfarm_learner

5. Create your Playwright project:
   mkdir playwright-assignment
   cd playwright-assignment

6. Confirm Node.js:
   node -v

7. Confirm npm:
   npm -v

8. Install Playwright:
   npm init playwright@latest

9. Confirm Playwright:
   npx playwright --version

10. List available tests:
   npx playwright test --list

11. Run a basic test:
   npx playwright test --project=chromium

12. Run headed mode if visual observation is needed:
   npx playwright test --project=chromium --headed

13. Generate an HTML report:
   npx playwright test --project=chromium --reporter=html

14. Open the report:
   npx playwright show-report
```

---

21. Important Notes

- Learners are expected to create and use their own Playwright project.
- Recommended learner project location:

```
C:\Users\talentfarm_learner\playwright-assignment
```

- The applications under test are located under:

```
C:\qa-lab\apps
```

- Use `127.0.0.1` when accessing the application from inside the VM.
- Use the VM public IP only when accessing the application from outside the VM.
- Use headed mode when you want to observe browser behavior.
- Use the HTML reporter when you need a browser-based report.
- Check `test-results` for screenshots, videos, traces, and failure artifacts.
- Do not delete preconfigured application or lab files unless instructed.
- Do not share screenshots or logs that expose passwords, tokens, or sensitive configuration values.
- Save screenshots and reports required for assignment evidence.
- Installed tools, created files, and test projects remain available after a normal VM restart.

---

22. Summary

In this quick tour, you learned:

- Where the lab applications are located
- How to open the applications under test
- How to create your own Playwright project
- How to install Playwright
- How to confirm Node.js, npm, and Playwright are available
- How to list and run Playwright tests
- How to run tests in headed mode
- How to generate and open the HTML report
- Where to find test results and application logs

Before beginning your assignment, make sure you can open the application in the browser, install Playwright in your own project folder, run a basic Playwright test, and view the test report.